

Service-Specific Enumeration & Web Attack-Surface Mapping

Previous Day: we explored an unfamiliar target and exposed what was on the machine. One of those reachable services was HTTP on port 3333. So for today, I want to explore what does HTTP on port 3333 actually do, and how would it actually be completed in a real security investigation.

With the scan, we obviously see 3333/tcp open http....but I want to transform that into a mental model of:

Client -> sends HTTP request -> goes to web server/application -> routes / resources -> application functionality -> user-supplied input (?) -> security boundaries -> questions worth investigating

Cold Retrieval

What is the difference between port enumeration and service detection?

Port enumeration is trying to identify reachable services on a computer, and then service detection actually identifies what version of an application is running behind them.

Why did finding HTTP on port 3333 not establish a vulnerability?

That was an exposure, we first would need to identify weaknesses that lead to a vulnerability.

Explain: Observation, Inference, Finding

If you receive an unfamiliar authorized IP, why shouldn't your first thought be: "Which exploit do I run?"

You need to learn the system; I think a first good step is to test reachability with ping, then attempt nmap scans.

HTTP: Understand What You're Actually Touching

Suppose you enter into the browser URL: http://TARGET_IP:3333/something

Breakdown:

- http://: the protocol that the client and server agree to use
- TARGET_IP: host communicating with
- :3333: TCP port on that host where the web service is listening
- /something: resource/path requesting from the application

Browser -----> web server / application -----> browser
TCP connection to TARGET:3333 HTTP response
HTTP request

Requests

A basic HTTP request has: method, path, headers, optional body

Two methods:

1. GET: usually retrieve something ...
 - a. GET /index.html
2. POST: submitting something
 - a. POST /login (potentially carrying data in the request body)

Responses

The server will respond to a request with: Status code, headers, body

Code	Meaning
200	Request succeeded
301/302	Redirect
403	Server refuses access
404	Requested resource not found
500	Server encountered internal error

Reconnect to Your Target

<https://tryhackme.com/room/vulniversity>

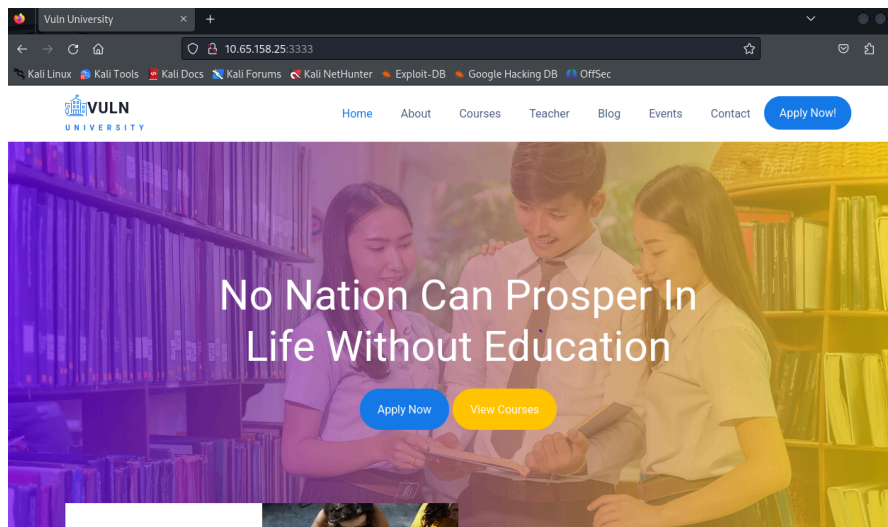
Redeploy machine, i'll test connectivity after it's giving me the IP

Target IP: 10.65.158.25

Ping command works fine, so the target machine is reachable.

Now i'm going to open the website...I can do this by going to the browser and typing

<http://10.65.158.25:3333>



I'm going to behave like a normal user and snoop through the website.

Purpose of website application: university website (vuln university)

Features:

Visible links/nav: nav bar at top of website + social media links and contact at the end

Forms/inputs: open ended form to type in a major you wish to study, enter email address to subscribe to newsletter

Authentication:

Other user actions: video on the homepage

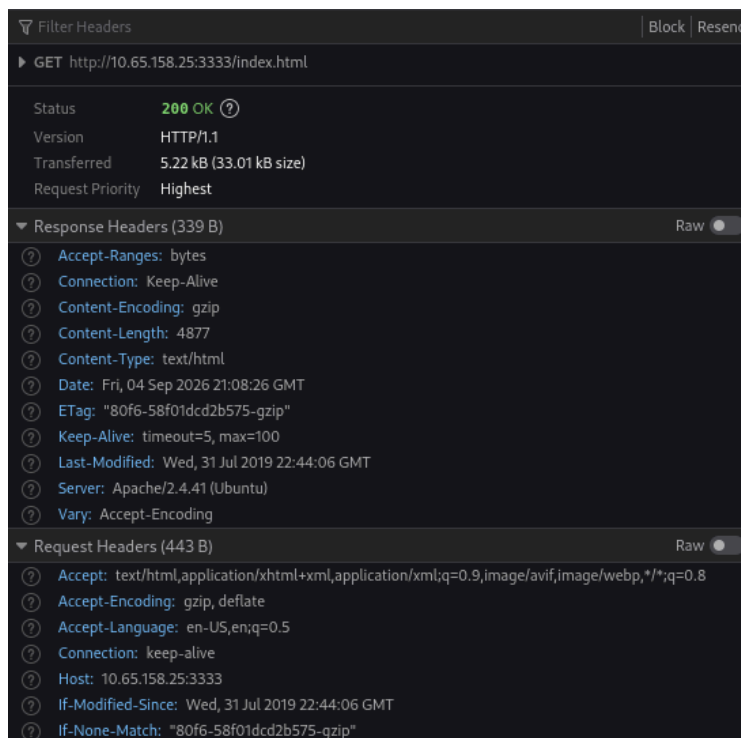
text/technology clues:

Note: per the exercise none of the website is really functional : (

Basically what I'm trying to do here is find what capabilities this website exposes to me (a remote user)

See HTTP Instead of Just Seeing a Website

In Developer Tools: Hit F12 then Network. Refresh the page and select the main request.



Request Method: GET

Request URL/path: index.html

Status code: 200

Response headers above^, Content-Type: text/html

Going to try clicking the home nav button within the application and watching what request appears....Eh, there's no real website functionality for me...Weird

Basically: browser action -> HTTP request -> application processing -> HTTP response

Learn curl as an HTTP Client

This is now a move from GUI to CLI. (command line interface). Curl is an HTTP/network client.

Run: **curl http://10.65.158.25:3333**

It is displaying the website's code basically. It is just the website, but in my terminal in a sense.

Try: **curl -i <http://10.65.158.25:3333>**

The -i flag includes response headers in the output.

Investigate headers

Server: what web server software is handling the HTTP request

Content-Type: what type of data the server is sending back

Content-Length: size of HTTP response body in bytes

Location: N/A

Set-Cookie: N/A

Routes and Resources: A Major Web Concept

The fact that: / exists does not mean: / is the entire application. There can be many paths, such as: /images/, /login/, /api/, /documents/, /admin/, /something-else/

Automated Content Discovery

You could manually guess: /login/, /admin/, /images/, etc... but that's pretty dumb if you ask me.

Content-discovery tools take a wordlist of candidate names and repeatedly ask the server: does /WORD exist?

Tool name: Gobuster

To see if installed: gobuster --help

```
(regis@kali)~$ gobuster --help
NAME:
  gobuster - the tool you love

USAGE:
  gobuster command [command options]

VERSION:
  3.0.2

AUTHORS:
  Christian Mehlmauer (@firefart)
  OJ Reeves (@TheColonial)

COMMANDS:
  dir      Uses directory/file enumeration mode
  vhost    Uses VHOST enumeration mode (you most probably want to use the IP address as the URL parameter)
  dns      Uses DNS subdomain enumeration mode
  fuzz     Uses fuzzing mode, Replaces the keyword FUZZ in the URL, Headers and the request body
  tftp     Uses TFTP enumeration mode
  s3       Uses aws bucket enumeration mode
  gcs      Uses gcs bucket enumeration mode
  help, h  Shows a list of commands or help for one command

GLOBAL OPTIONS:
  --help, -h      show help
  --version, -v   print the version
```

Shape of command: **gobuster dir -u http://TARGET_IP:333 3 -w wordlist_path**

Wordlists

On kali: /usr/share/wordlists/

Point of a wordlist is: provide plausible path names for the tool to request

Using a fast directory discovery tool called **Gobuster**, you will locate a directory to which you can use to upload a shell.

Let's first start by scanning the website to find any hidden directories. To do this, we're going to use **Gobuster**.



Gobuster is a tool for brute-forcing URIs (directories and files), DNS subdomains, and virtual host names. For this machine, we will focus on using it to brute-force directories.

Download Gobuster [here](#), or if you're on Kali Linux run **sudo apt-get install gobuster**.

To get started, you will need a wordlist for Gobuster (which will be used to quickly go through the wordlist to identify if a public directory is available. If you use [Kali Linux](#), you can find many wordlists under **/usr/share/wordlists**. You can also use the [wordlist](#) for directories located at **/usr/share/wordlists/dirbuster/directory-list-1.0.txt** in the AttackBox.

Now let's run Gobuster with a wordlist using **gobuster dir -u http://10.65.158.25:3333 -w**.

Gobuster flag	Description
-e	Print the full URLs in your console
-u	The target URL
-w	Path to your wordlist
-U and -P	Username and Password for Basic Auth
-p <x>	<u>P</u> roxy to use for requests
-c <http cookies>	Specify a cookie for simulating your auth

```
(nigist@kaliN)-[~]
$ gobuster dir -u http://10.65.158.25:3333 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://10.65.158.25:3333
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

images (Status: 301) [Size: 320] [→ http://10.65.158.25:3333/images/]
css (Status: 301) [Size: 317] [→ http://10.65.158.25:3333/css/]
js (Status: 301) [Size: 316] [→ http://10.65.158.25:3333/js/]
fonts (Status: 301) [Size: 319] [→ http://10.65.158.25:3333/fonts/]
internal (Status: 301) [Size: 322] [→ http://10.65.158.25:3333/internal/]
Progress: 34353 / 220559 (15.58%)^C
```

← → ↺ 🏠 10.65.158.25:3333/internal/

Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB OffSec

Upload

Browse...

No file selected.

Submit